

AI 机器人课程 Week 14 小组项目报告书

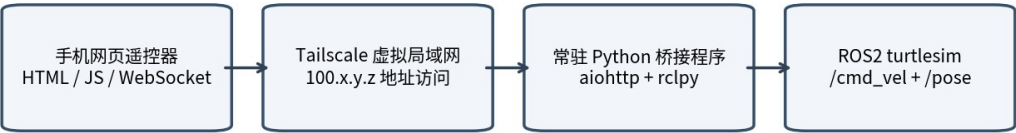
方向 B：手机遥控 + turtlesim 迷宫自动探索

完成形式	单人项目
姓名/学号	백하 / 20231913
项目方向	方向 B：turtlesim 二维平面 ROS2 乌龟
个人分工	网页遥控、Tailscale/ROS2 链路、桥接程序、迷宫生成、A* 自动探索、测试与报告
提交材料	代码、演示视频 turtlesim_auto.mp4、PDF 报告书

日期：2026 年 6 月

一、项目概述与系统链路

本项目选择 Week 14 方向 B。目标是把手机网页、Tailscale 虚拟局域网、Python 常驻桥接程序、ROS2 turtlesim 和迷宫自动探索整合成一条完整控制链路。手机网页负责输入和状态显示；桥接程序负责 WebSocket 通信、ROS2 指令发布、位姿订阅、碰撞预测与状态广播；turtlesim 根据 /turtle1/cmd_vel 执行动作，并通过 /turtle1/pose 回传位姿。



控制链路：手机输入 -> 网络传输 -> 桥接解析 -> ROS2 速度发布；状态通过 WebSocket 回传网页

图1 手机 -> Tailscale -> 桥接程序 -> ROS2 turtlesim 控制链路

系统采用“单一常驻程序”原则：网络接收和机器人控制均放在 turtlesim_web_bridge.py 中，不额外启动第二个控制程序，避免多个进程抢占同一控制链路。网页端通过 JSON 命令控制手动/自动、复位和停止，后端持续返回位置、朝向、碰撞原因、目标状态和实际速度。

模块	文件/接口	作用
网页遥控器	index.html	方向键、自动/手动按钮、二维迷宫、轨迹和遥测显示
网络通信	WebSocket /ws	发送 command/auto/reset/stop，接收 state 状态包
桥接程序	turtlesim_web_bridge.py	aiohttp 服务 + rclpy 节点；发布 Twist，订阅 Pose
自动探索	explorer.py + maze.py	A* 路径规划，按 waypoint 输出线速度与角速度

二、迷宫设计与安全规则

maze.py 使用递归回溯生成 6×6 网格迷宫，再通过 braid 随机打通部分墙体形成环路。默认参数下生成 20 个障碍矩形、40 条开放边，其中 5 条为额外环路；程序自带 BFS 自检，确认从起点到终点可达。碰撞检测按乌龟半径 0.45 对边界和障碍物做膨胀，若预测下一步会越界或撞墙，就把线速度置为 0 并返回 blocked/reason。

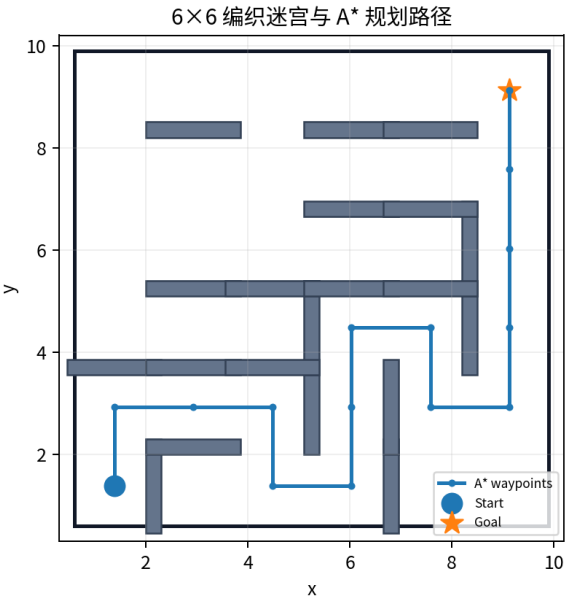


图2 6×6 编织迷宫与 A* 路径示意

三、代码改动说明

文件	关键函数/内容	说明
maze.py	build_maze(), neighbors(), _self_check()	生成迷宫、起终点、障碍矩形和邻接关系；验证地图可达
explorer.py	astar(), Planner.decide()	用 A* 求最短格点路径，再把路径点转换为连续运动命令
turtlesim_web_bridge.py	TurtleWebBridge, compute_safe_motion()	常驻桥接节点；处理网页命令、ROS2 发布/订阅、碰撞与终点判定
index.html	setAutoMode(), drawMaze(), updateTelemetry()	手机端 UI；支持方向键、自动模式、复位、轨迹绘制与状态显示

四、自动探索策略

自动模式由网页发送 {type:"auto", enabled:true} 触发。桥接程序重新创建 Planner，并在每个控制周期调用 decide(state)。Planner 先把当前位姿映射到迷宫格子，再用 A* 规划到终点的格点路径；执行阶段逐个追踪 waypoint。若朝向误差大于 TURN_FIRST，则先原地转向；误差足够小时再前进。到达当前 waypoint 后切换到下一个点，进入终点圆形区域后停止。

```
A* 核心逻辑：
openq = [(0, start)]
came = {start: None}; g = {start: 0}
for nxt in neighbors(current):
    next_cost = g[current] + 1
    if nxt not in g or next_cost < g[nxt]:
        g[nxt] = next_cost
        came[nxt] = current
    push(next_cost + manhattan(nxt, goal), nxt)
```

A* 的优点是能利用迷宫邻接关系直接求较短路径，适合本项目加入环路后的复杂迷宫。和简单右手法则相比，它不依赖墙面方向，也不容易在环路中重复绕行。连续控制层仍保留碰撞预测，因此即使自动算法输出前进速度，最终执行前也会进行安全校验。

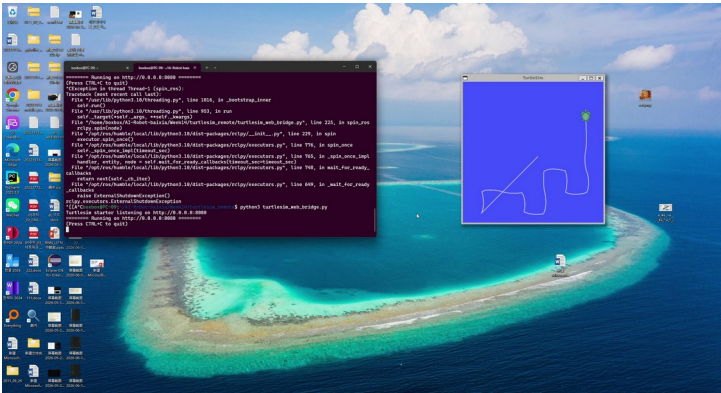


图3 自动探索演示视频截帧

五、测试结果

- 手机网页能够完成前进、后退、左转、右转、停止、复位和自动模式切换。
- 网页画布能实时显示迷宫墙体、起点、终点、乌龟位置、朝向和运动轨迹。
- 后端能够识别 boundary、obstacle、goal_reached 等状态，并把目标速度与实际速度回传到遥测面板。
- 自动模式能够按 A* 路径追踪 waypoint，并在进入终点区域后停车。
- 补充测试截图：图 4 展示了本项目在手机浏览器中的真实运行界面，能够看到已连接状态、手动/自动模式切换按钮、方向控制区以及二维迷宫预览。



• 图 4 手机端真实运行截图

六、问题与解决

手机打不开网页：使用 Tailscale 让手机和电脑进入同一虚拟局域网，并通过 http://<Tailscale_IP>:8080 访问。

松手后仍继续运动：在 touchend、mouseup、mouseleave 事件中统一发送 stop，避免命令持续生效。

离散路径难以直接执行：把格点路径转换为 cell center waypoint，并设置 REACH_TOL 到达阈值。

自动转弯不够平滑：采用角度误差比例控制，误差较大时先转向，误差较小时再前进。

七、成果与单人分工

本项目已完成方向 B 的核心要求：手机控制、Tailscale/局域网访问、WebSocket 双向通信、ROS2 turtlesim 控制、二维迷宫碰撞规则、自动探索算法、路径轨迹显示和演示视频。由于是单人项目，所有模块均由本人独立完成，具体分工如下：

任务	完成情况
链路打通	安装/运行 ROS2 turtlesim，启动常驻桥接程序，手机访问 8080 网页控制器
网页与交互	实现方向键、停止、复位、自动模式、迷宫预览和遥测面板
迷宫与碰撞	实现 6×6 编织迷宫、边界检测、障碍膨胀碰撞、终点判定
自动探索	实现 A* 搜索、waypoint 跟踪、转向控制和到达终点停车
测试展示	录制 turtlesim_auto.mp4，并整理报告书

八、总结

通过本项目，我把网页前端、虚拟局域网、Python WebSocket 服务、ROS2 节点通信和路径搜索算法整合成一个可运行的机器人远程操控系统。项目的关键收获是理解“网页只发命令、桥接程序统一接管机器人控制”的工程结构，以及如何把离散 A* 路径转换成连续速度控制。后续可继续加入用时统计、碰撞次数统计、多算法对比和网页端迷宫参数配置。

参考资料：AI 机器人课程 Week 14 项目要求；本项目代码文件 maze.py、explorer.py、turtlesim_web_bridge.py、index.html、requirements.txt。